Sprint 9 · Reto «Conocer al huésped»

Ficha completa del reto, para consultar. Para trabajar, el reto en una página. Qué se sube y cómo se evalúa: entrega y rúbrica.

- Volver al README del sprint.
- Si es vuestro primer reto: Cómo se trabaja un reto.

RETO Ficha del reto

1 Qué os pide el cliente

Qué os pide el cliente. Lucía, la directora comercial de TurisData Canarias, envía **la misma newsletter a sus 1.200 huéspedes** y vende sus **25 experiencias** con los mismos banners. Sospecha que hay «tipos» de cliente muy distintos y que cada uno querría cosas diferentes. Os pide: *«Decidme quiénes son mis clientes, cómo los llamo y qué le enseño a cada uno.»* Ninguna columna dice a qué tipo pertenece cada huésped: vosotros tenéis que **descubrir los grupos** con *clustering* (agrupar datos parecidos sin respuestas previas), **dibujarlos** en un mapa 2D, **ponerles nombre** y proponer **una acción comercial por grupo**. Además, construiréis un **recomendador** sencillo: dado un huésped, sugerirle experiencias que probablemente le gusten.

Cómo se trabaja. No hace falta experiencia previa: los cuadernos del sprint (`S09_01` , `S09_02` y `S09_03`) ya os hacen practicar cada pieza, y el reto consiste en **reutilizarlas con vuestras propias decisiones** y, sobre todo, en **explicar** qué habéis encontrado. No hay pruebas automáticas ni solución del docente: los grupos se validan con **sentido de negocio** y con **métricas internas** (silueta, RMSE). En clase tenéis 10 minutos de planning y 45 de reto el día 1, y 50 de reto más 40 de entrega, review y retrospectiva el día 2; lo que no quepa va a casa. El equipo decide hasta dónde llega: mínimo, estándar o extra.

2 · Recursos del reto

Recurso	Para qué lo usaréis	Dónde está
Python con `numpy` , `pandas` , `matplotlib` / `seaborn` y `scikit-learn` (solo esas bibliotecas: nada de librerías de recomendación especializadas)	Escribir el cuaderno del reto	Ya instaladas en Google Colab
Google Colab	Crear y ejecutar `reto_S09_conocer_al_huesped.ipynb`	En colab.research.google.com: *Archivo → Cuaderno nuevo*
Los datos de huéspedes `huespedes.csv`	1.200 huéspedes con 7 variables (`edad` , `gasto_medio_dia` , `noches_medias` , `reservas_anio` , `antelacion_media` , `pct_gasto_actividades` , `viaja_con_ninos`). Son **sintéticos**	No hace falta descargarlos: la celda «Preparación» de los cuadernos `S09_01` – `S09_03` los baja sola
Las valoraciones `valoraciones_experiencias.csv` y `experiencias.csv`	2.232 valoraciones (de 1 a 5 estrellas) de 300 usuarios sobre 25 experiencias, y el catálogo con nombre, categoría e isla	Igual: los baja la celda «Preparación»
Cuadernos del sprint: `S09_01` , `S09_02` y `S09_03`	El código de cada paso: escalar, k-means y silueta, PCA, recomendadores, RMSE y precision@3	Colab; su código explicado

Recurso	Para qué lo usaréis	Dónde está
Presentaciones Clustering y PCA y Sistemas de recomendación	Repasar los conceptos	En el repositorio del curso
La plantilla de la ficha `plantilla_ficha_segmentos.md`	Ficha de segmentos con huecos, nota de IA responsable y sección «Revisión con la rúbrica»	En GitHub: abrid el archivo y copiad el texto con el botón *Raw*
Cómo entregar en GitHub	Subir el cuaderno y la ficha a `sprint-09/`	Guía de entrega
Autoevaluación y coevaluación	Cada persona rellena su hoja	Hoja del sprint
Un asistente de IA (opcional)	Entender conceptos, pedir nombres y pensar riesgos	Con las reglas de la «Ayuda 1» del paso a paso

3 · Guía paso a paso

Son cinco pasos, de **105 minutos en total** (unos 45 el día 1 y unos 60 el día 2; el aula da 45 y 50, así que lo que no quepa va a casa). Cada uno dice **qué hacéis** (la lista corta), **cómo se hace** con el detalle, **qué deberíais ver** y **cuándo está hecho**. Las ayudas (IA, errores, atascos) están al final. Si el tiempo aprieta, completad el nivel mínimo y dejad lo demás para casa.

Paso 1 · Preparar el cuaderno y los datos (día 1, 10 minutos)

1. Crear en Colab el cuaderno `reto_S09_conocer_al_huesped.ipynb`.
2. Cargar `huespedes.csv` y mirar sus columnas.
3. Escalar las variables numéricas.
4. Repartir el trabajo de los pasos 2 y 3.

En sencillo. Lucía, la directora comercial, manda la misma newsletter a sus 1200 huéspedes y quiere saber qué tipos de cliente tiene. Antes de buscar grupos hay que **preparar la mesa de trabajo**: abrir un cuaderno nuevo, traer los datos y dejarlos en condiciones. El paso más importante es **escalar**. Imaginad que comparáis a dos personas por su edad (en años) y por lo que gastan (en euros): como los euros son números mucho más grandes, el gasto mandaría y la edad apenas contaría. Escalar es pasar todo a una misma regla para que cada columna pese igual. No hace falta saber Python: cada línea que escribáis viene explicada y, si copiáis el código tal cual, funciona.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Cuaderno y celda	Un cuaderno de Colab es un documento hecho de **celdas**. Las de código se ejecutan con ▶ (o `Shift + Enter`) y debajo muestran el resultado.
Variable	Un nombre que guarda algo para usarlo después. En `X = 5`, `X` guarda el 5.
DataFrame	Una tabla de datos dentro de Python, con filas y columnas. Aquí, `huespedes`: un huésped por fila.
Función	Una mini-receta con nombre. `dato("huespedes.csv")` le pide la receta `dato` el fichero de ese nombre.
Escalar (estandarizar)	Restar a cada columna su media y dividir por su desviación, para que todas tengan media 0 y la misma escala.
Matriz `X`	Una tabla solo de números, sin nombres de columna. Es lo que usan los algoritmos de agrupar.
Semilla (`SEMILLA`)	Un número fijo (aquí, 42) que hace que lo «aleatorio» salga igual cada vez. Sin ella, dos personas obtendrían resultados un poco distintos.
`.shape`	Las dimensiones de una tabla: `(1200, 7)` es 1200 filas y 7 columnas.

Cómo se hace

1. **Crear el cuaderno.** Entrad en colab.research.google.com, *Archivo → Cuaderno nuevo*. Arriba a la izquierda pulsad sobre el nombre y cambiadlo por `reto_S09_conocer_al_huesped.ipynb`. Trabajad en una sola pantalla compartida o pulsad **Compartir** (arriba a la derecha) y dad permiso de **Editor** al resto. **Nunca dos personas en la misma celda a la vez.**
2. **Primera celda: la preparación.** Pulsad en la celda de código que Colab ya trae, pegad esto y pulsad ▶:

```
import os, urllib.request

BASE_URL = "https://raw.githubusercontent.com/cCoolBoox/IFCD107-alumnado/main/datos/"

def dato(nombre):
    """Devuelve la ruta local de un archivo de datos del curso (lo descarga si no está)."""
    for carpeta in ("datos", "../datos", "../../datos"):
        ruta = os.path.join(carpeta, nombre)
        if os.path.exists(ruta):
            return ruta
    os.makedirs("datos", exist_ok=True)
    ruta = os.path.join("datos", nombre)
    urllib.request.urlretrieve(BASE_URL + nombre, ruta)
    return ruta
```

   Qué hace cada línea:

   - `import os, urllib.request`: carga dos herramientas de Python, una para trabajar con carpetas y otra para descargar de internet.
   - `BASE_URL = "…"`: guarda la dirección web donde están los datos del curso.
   - `def dato(nombre):`: crea la función `dato`. El resto de líneas (con sangría, es decir, empujadas a la derecha) son lo que hace: busca el fichero en el ordenador y, si no está, lo descarga. **Copiadla entera y sin cambiar nada.**

   No muestra nada: si no sale un error en rojo, está bien. **Esta celda se ejecuta siempre la primera**, también cuando abráis el cuaderno otro día.
3. **Segunda celda: herramientas, semilla y datos.** Pulsad **+ Código**, pegad y pulsad ▶:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA

SEMILLA = 42
np.random.seed(SEMILLA)
sns.set_theme(style="whitegrid")

huespedes = pd.read_csv(dato("huespedes.csv"))
print(huespedes.shape)
huespedes.head()
```

   Qué hace cada línea:

   - Las líneas con `import` traen las herramientas que usaréis en todo el reto: `numpy` y `pandas` (números y tablas), `matplotlib` y `seaborn` (gráficos), y `scikit-learn` (los algoritmos: escalar, agrupar, silueta y PCA).
   - `SEMILLA = 42` y `np.random.seed(SEMILLA)`: fijan la semilla. **Usad siempre `SEMILLA`** cuando una función pida `random_state`.
   - `huespedes = pd.read_csv(dato("huespedes.csv"))`: lee el fichero y guarda la tabla en la variable `huespedes`.
   - `print(huespedes.shape)`: escribe las dimensiones. La última línea, `huespedes.head()`, enseña las cinco primeras filas.

4. **Mirar las columnas.** En otra celda escribid `huespedes.describe().round(1).T` y pulsad ▶. Es un resumen de cada columna (`mean` es la media, `min` y `max` los extremos). Fijaos en dos cosas: `id_huesped` es solo un número de identificación, y las otras **siete** son números de comportamiento (`viaja_con_ninos` solo vale 0 o 1). Y fijaos en las **escalas**: la edad va de 18 a 75 y el gasto diario, de 35 a 340.
5. **Escalar.** Nueva celda:

   ```
   variables = [c for c in huespedes.columns if c != "id_huesped"]
   X = StandardScaler().fit_transform(huespedes[variables])
   print(X.shape, round(X.mean(), 3), round(X[:, 0].std(), 3))
   ```

   Qué hace cada línea:

   - `variables = [c for c in huespedes.columns if c != "id_huesped"]`: hace una lista con los nombres de todas las columnas **menos** `id_huesped` (el identificador no describe a nadie y no debe entrar). `!=` significa «distinto de».
   - `X = StandardScaler().fit_transform(huespedes[variables])`: coge esas siete columnas, las escala y guarda el resultado en `X`. **Guardad siempre en `X`**: los pasos 2 y 3 lo usan.
   - `print(…)`: escribe las dimensiones de `X`, su media global y la desviación de su primera columna, para comprobar que se escaló bien.
6. **Repartir el trabajo de los pasos 2 y 3.** Antes de seguir, apuntad en el tablero quién hace qué. Un reparto posible:

Persona	Tarea	Paso
A	Probar k-means con `k` de 2 a 8, silueta y elegir `k`	2
B	Mapa PCA y perfil de cada segmento	3
C	Nombres y acciones comerciales (con A y B)	3

   Si sois más de tres, la persona que quede libre empieza a leer el paso 4. Un cuaderno compartido solo funciona si cada persona escribe en **sus** celdas.

Qué deberíais ver. Tras la segunda celda, `(1200, 8)` y una tabla de cinco filas que empieza por `id_huesped` 1 (con 26 años), 2 (41), 3 (60)… Tras `describe()`, una tabla con una fila por columna (la media de la edad es 41,2 y la del gasto diario, 124,2). Y tras el escalado, `(1200, 7) 0.0 1.0`: una matriz `X` de 1200 filas y 7 columnas con la media cerca de 0.

Hecho cuando: `X` existe y está escalada, y cada persona sabe qué parte del paso 2 o del 3 le toca.

Si algo falla

Veis…	Qué significa	Qué hacer
`NameError: name 'dato' is not defined` (o `'pd'`, `'X'`)	No se ha ejecutado la celda donde se define, o Colab se reinició y olvidó todo.	*Entorno de ejecución* → *Ejecutar todo*, o pulsad ▶ en las celdas de arriba, **en orden**.
`HTTPError: HTTP Error 404: Not Found`	El nombre del fichero está mal escrito.	Es exactamente `"huespedes.csv"`: en minúsculas y con la `p` y la `u`.
`SyntaxError: invalid character '“' (U+201C)`	Hay comillas «rizadas», típicas al copiar de un documento.	Usad comillas rectas `"` y volved a escribirlas.
`SyntaxError: '(' was never closed`	Falta un paréntesis `)` al final de la línea.	Contad los paréntesis: cada `(` tiene su `)`.
`KeyError: "None of [Index(['edades'], dtype='object')] are in the [columns]"`	El nombre de una columna está mal escrito.	Copiadlo tal cual de `huespedes.columns` (por ejemplo, `edad` sin la `es`).
`X.shape` sale `(1200, 8)`	`id_huesped` se ha colado en `X`.	Escalad `huespedes[variables]`, que ya no lo lleva.

Paso 2 · Segmentar y elegir k (día 1, 20 minutos)

1. Probar k-means con `k` de 2 a 8.
2. Calcular la silueta de cada `k` y dibujar la curva.
3. Elegir `k` con una razón técnica **y** una de negocio.
4. Escribir esa razón en una frase.

En sencillo. Imaginad una cena con 1200 invitados y que hay que sentarlos en mesas según lo parecidos que sean. Antes de empezar tenéis que decidir **cuántas mesas** (`k`). K-means coloca un «centro» en cada mesa y manda a cada persona al centro más cercano; después recoloca los centros y repite. Pero ¿cuántas mesas son las buenas? Aquí **no hay respuesta correcta en los datos**: probáis de 2 a 8 mesas y medís con la **silueta** qué tan compactas y separadas quedan (cuanto más alta, mejor). El número más alto no decide solo: también hay que preguntarse si Lucía puede **hacer algo distinto** para cada grupo. No hace falta saber cómo funciona k-means por dentro: basta con saber qué se le pide y qué devuelve.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
k-means	Un algoritmo que reparte los datos en `k` grupos de elementos parecidos, sin que nadie le diga de antemano a qué grupo pertenece cada uno.
`k`	El número de grupos. Lo decidís vosotros.
Etiqueta	El número de grupo que el algoritmo asigna a cada huésped (0, 1, 2…). Aquí, `etiquetas` tiene 1200 números.
Silueta	Una nota de −1 a 1 para cada `k`: cuanto más alta, grupos más compactos y más separados entre sí. Por encima de 0,5 es una estructura buena; entre 0,25 y 0,5, aceptable.
Diccionario (`{}`)	Una agenda de parejas `clave: valor`. Aquí, `siluetas` guarda `k: silueta`.
Bucle `for`	Repite unas líneas para cada valor de una lista: `for k in range(2, 9)` las repite para k = 2, 3… 8.
`range(2, 9)`	Los números del 2 al 8: el último (el 9) **no entra**.
`n_init=10`	Hace que k-means se ejecute 10 veces desde puntos de partida distintos y se quede con la mejor. Resultado más estable.
`max(…, key=…)`	Devuelve la clave del diccionario cuyo valor es el mayor.

Cómo se hace

1. **Probar cada `k` y guardar su silueta.** En una celda nueva (pulsad **+ Código**):

```
siluetas = {}
for k in range(2, 9):
    etiquetas = KMeans(n_clusters=k, n_init=10, random_state=SEMILLA).fit_predict(X)
    siluetas[k] = silhouette_score(X, etiquetas)
k_mejor = max(siluetas, key=siluetas.get)
print({k: round(v, 3) for k, v in siluetas.items()})
print("k con mejor silueta:", k_mejor)
```

   Qué hace cada línea:

   - `siluetas = {}`: crea un diccionario vacío, la agenda donde se anotará cada resultado.
   - `for k in range(2, 9):`: repite las líneas de debajo (las que están **con sangría**, empujadas a la derecha) para cada `k`. Los dos puntos del final son obligatorios.
   - `KMeans(n_clusters=k, n_init=10, random_state=SEMILLA).fit_predict(X)`: prepara k-means con `k` grupos, lo ejecuta sobre `X` y devuelve las `etiquetas`. `random_state=SEMILLA` fija el azar.
   - `siluetas[k] = silhouette_score(X, etiquetas)`: calcula la silueta media y la anota en la agenda con la clave `k`.
   - `k_mejor = max(siluetas, key=siluetas.get)`: guarda el `k` de silueta más alta.
   - Las dos líneas `print` escriben la agenda (redondeada a 3 decimales) y el mejor `k`.

   Pulsad ▶ (tarda unos segundos). Sale `{2: 0.367, 3: 0.415, 4: 0.431, 5: 0.414, 6: 0.326, 7: 0.317, 8: 0.257}` y `k con mejor silueta: 4`.

2. **Dibujar la curva.** Otra celda:

```
fig, ax = plt.subplots(figsize=(6, 3.5))
ax.plot(list(siluetas), list(siluetas.values()), marker="o")
ax.axvline(k_mejor, color="tab:red", linestyle="--", label=f"mejor k = {k_mejor}")
ax.set_xlabel("Número de grupos (k)")
ax.set_ylabel("Silueta media")
ax.set_title(f"La silueta es más alta con k = {k_mejor}")
ax.legend()
plt.show()
```

   Qué hace cada línea:

   - `fig, ax = plt.subplots(figsize=(6, 3.5))`: crea el lienzo del gráfico (6 × 3,5 pulgadas). `ax` es el dibujo dentro del lienzo.
   - `ax.plot(…, marker="o")`: dibuja la línea con un punto en cada `k` (eje horizontal) y su silueta (eje vertical).
   - `ax.axvline(k_mejor, …)`: añade una línea vertical roja y discontinua en el mejor `k`.
   - `set_xlabel`, `set_ylabel`, `set_title`: ponen los nombres de los ejes y el título. El título dice una **conclusión**, no solo «silueta».
   - `ax.legend()` y `plt.show()`: muestran la leyenda y el gráfico.

   Veréis una curva que sube hasta k = 4 y baja después, con la línea roja en 4.

3. **Mirar cuántos huéspedes hay en cada grupo.** La silueta de 3, 4 y 5 casi empata (0,415, 0,431 y 0,414), así que hay que mirar el negocio. Otra celda:

```
for k in (3, 4, 5):
    etiquetas = KMeans(n_clusters=k, n_init=10, random_state=SEMILLA).fit_predict(X)
    print(k, "grupos · tamaños:", np.bincount(etiquetas))
```

   Qué hace cada línea:

   - `for k in (3, 4, 5):` : repite para esos tres valores, escritos entre paréntesis.
   - `np.bincount(etiquetas)` : cuenta cuántos huéspedes hay con la etiqueta 0, con la 1, con la 2…

   Sale: `3 grupos · tamaños: [317 480 403]`, `4 grupos · tamaños: [194 402 317 287]` y `5 grupos · tamaños: [ 72 402 317 126 283]`. Con 5 grupos, uno tiene solo 72 huéspedes (el 6 %): cuesta justificar una campaña para tan pocos. Con 4, el más pequeño tiene 194 (el 16 %) y los cuatro son lo bastante grandes para actuar.

4. **Elegir `k` y escribir la frase con las dos razones.** En una celda de texto (**+ Texto**) con este molde:

```
Elegimos k = … porque la silueta es … y, para el negocio, … grupos se pueden gestionar (el más pequeño tiene el … % de
los huéspedes).
```

   Por ejemplo, con los números de esta ejecución: *«Elegimos k = 4 porque su silueta es la más alta (0,431), aunque 3 y 5 casi empatan, y, para el negocio, 4 grupos se pueden gestionar: el más pequeño tiene el 16 % de los huéspedes.»* Los vuestros pueden ser otros. Vale elegir 3 o 5 si lo razonáis. **No vale** elegir solo porque «salía el número más alto» ni «porque lo dijo la IA». Esta frase irá también a la cabecera de la ficha del paso 5.

Qué deberíais ver. Una agenda con siluetas para `k` = 2 a 8 (la mayor, en torno a 0,43 con `k` = 4), una curva con la línea roja en `k_mejor`, los tamaños de los grupos para 3, 4 y 5, y un `k` elegido entre 3 y 5 con su frase. Si usáis otra versión de scikit-learn, las siluetas pueden cambiar en las milésimas.

Hecho cuando: hay curva de silueta, un `k` elegido y la frase con las dos razones.

Si algo falla

Veis…	Qué significa	Qué hacer
`NameError: name 'X' is not defined` (o `'KMeans'`)	No se han ejecutado las celdas del paso 1, o Colab se reinició.	Pulsad ▶ en las celdas de arriba, en orden (*Entorno de ejecución → Ejecutar todo*).
`ValueError: Number of labels is 1. Valid values are 2 to n_samples - 1 (inclusive)`	Habéis probado `k = 1` (`range(1, 9)`): con un solo grupo no hay silueta.	Escribid `range(2, 9)` .
La curva acaba en 7	Habéis escrito `range(2, 8)` : el último número no entra.	Escribid `range(2, 9)` para llegar al 8.
`IndentationError: expected an indented block after 'for' statement on line 2` o `SyntaxError: expected ':'`	Falta la sangría en las líneas de dentro del bucle, o los dos puntos del final.	Dejad 4 espacios delante de las líneas de dentro y poned `:` al final de la línea del `for` .
La silueta sale más alta (por ejemplo, 0,52 con `k` = 2)	Habéis agrupado sin escalar (`huespedes[variables]` en vez de `X`). Engaña: mandan las columnas de números grandes, como el gasto.	Usad `X` , la matriz escalada del paso 1.
`k_mejor` es 3 o 5, o las cifras no coinciden con las de aquí	Con otra versión de scikit-learn cambian unas milésimas; con 3, 4 y 5 casi empatados, puede ganar otro.	Es válido. Usad **vuestras** cifras en la frase y la razón de negocio.

Paso 3 · Mapa PCA e interpretación de los segmentos (día 1, 15 minutos)

1. Dibujar el mapa 2D con PCA, con los segmentos coloreados.
2. Calcular el perfil medio de cada segmento y su tamaño.
3. Ponerle nombre a cada segmento, con cifras.
4. Proponer una acción comercial por segmento.

En sencillo. Cada huésped tiene siete datos, y un dibujo solo tiene dos ejes. **PCA** es como hacer una foto a una escultura desde el mejor ángulo: se pierde algo (la profundidad), pero se ve la forma. Así pintaréis los 1200 huéspedes en un mapa y comprobaréis a simple vista si los grupos están separados. Después, para cada grupo calcularéis su **perfil**: la media de cada dato, como el retrato robot de un cliente típico. Con ese retrato, el grupo recibe un **nombre** que Lucía entienda y una **acción comercial**. El nombre tiene que salir de **vuestras cifras**, no del ingenio. No hace falta saber cómo calcula PCA las componentes: lo hace una línea de código.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
PCA	Una técnica que resume muchas columnas en pocas «columnas nuevas» (componentes) para poder dibujar los datos.
Componente	Cada eje nuevo del mapa. La componente 1 recoge la mayor parte de la variación; la 2, la siguiente.
% de varianza	Cuánta información del conjunto conserva cada componente. Es lo que va en los ejes del mapa.
Segmento	Un grupo de huéspedes parecidos entre sí. Aquí, cada número de `grupo`.
Perfil	La media de cada variable dentro de un segmento: el retrato del cliente típico de ese grupo.
`groupby`	Reparte las filas de una tabla por grupos y calcula algo (la media, por ejemplo) en cada uno.
Mapa de calor	Una tabla pintada: rojo si el grupo está por encima de la media, azul si está por debajo.
Acción comercial	Qué enseñarle a ese segmento, cuándo y por qué canal.

Cómo se hace

1. **Entrenar k-means con el `k` elegido y guardar el grupo de cada huésped.** Nueva celda (cambiad el 4 por el `k` que elegisteis):

   ```
   k_elegido = 4
   km = KMeans(n_clusters=k_elegido, n_init=10, random_state=SEMILLA).fit(X)
   huespedes["grupo"] = km.labels_
   print(huespedes["grupo"].value_counts().sort_index())
   ```

   Qué hace cada línea:

   - `km = KMeans(…).fit(X)`: entrena k-means y guarda el modelo en `km`. Es lo mismo del paso 2, pero ahora con un solo `k`.
   - `huespedes["grupo"] = km.labels_`: añade a la tabla una columna `grupo` con el número de grupo de cada huésped.
   - `value_counts().sort_index()`: cuenta cuántos huéspedes hay en cada grupo y los ordena por número.

   Pulsad ▶. Con `k` = 4 salen los grupos 0, 1, 2 y 3 con 194, 402, 317 y 287 huéspedes.

2. **El mapa PCA.** Nueva celda:

```
pca = PCA(n_components=2, random_state=SEMILLA)
Z2 = pca.fit_transform(X)
var1, var2 = pca.explained_variance_ratio_ * 100

fig, ax = plt.subplots(figsize=(7, 5))
dibujo = ax.scatter(Z2[:, 0], Z2[:, 1], c=huespedes["grupo"], cmap="tab10", s=10, alpha=0.7)
ax.set_title("Cuatro tipos de huésped bastante separados")
ax.set_xlabel(f"Componente 1 ({var1:.0f} % de la varianza)")
ax.set_ylabel(f"Componente 2 ({var2:.0f} % de la varianza)")
ax.legend(*dibujo.legend_elements(), title="Segmento")
fig.savefig("mapa_pca.png", dpi=150, bbox_inches="tight")
plt.show()
print(f"Información conservada con 2 componentes: {var1 + var2:.1f} %")
```

Qué hace cada línea:

- `PCA(n_components=2, …).fit_transform(X)`: resume las 7 columnas de `X` en 2 y guarda un par de coordenadas por huésped en `Z2`.
- `var1, var2 = pca.explained_variance_ratio_ * 100`: el % de varianza de cada componente, en dos variables.
- `ax.scatter(Z2[:, 0], Z2[:, 1], c=huespedes["grupo"], …)`: dibuja un punto por huésped. `Z2[:, 0]` es la primera columna (eje horizontal) y `Z2[:, 1]`, la segunda (eje vertical). `c=` colorea por grupo.
- `set_title(…)`: el título debe ser una **conclusión** («Cuatro tipos de huésped bastante separados»). **Cambiadlo** si en vuestro mapa no es verdad.
- `set_xlabel`, `set_ylabel`: nombres de los ejes, con el % de varianza escrito automáticamente. `ax.legend(…)` muestra los segmentos.
- `fig.savefig("mapa_pca.png", …)`: guarda el mapa como imagen, para la ficha del paso 5.

Se ve un punto por huésped en cuatro nubes de colores. Los ejes dicen 43 % y 26 % de la varianza y la última línea escribe `Información conservada con 2 componentes: 68.7 %`. Es una «foto» que conserva cerca del 69 %: si vuestro mapa sale mezclado, decidlo en la ficha. Aquí, dos de las nubes se tocan un poco.

3. **El perfil de cada segmento.** Nueva celda:

```
perfil = huespedes.groupby("grupo")[variables].mean().round(1)
perfil["n_huespedes"] = huespedes["grupo"].value_counts().sort_index()
perfil["pct"] = (perfil["n_huespedes"] / len(huespedes) * 100).round(0)
perfil
```

Qué hace cada línea:

- `huespedes.groupby("grupo")[variables].mean().round(1)`: agrupa por `grupo` y calcula la media de cada variable con un decimal.
- `perfil["n_huespedes"] = …` y `perfil["pct"] = …`: añaden el tamaño de cada grupo y su porcentaje sobre los 1200.

Pulsad ▶. Con `k` = 4 sale esta tabla (aquí, una fila por variable y una columna por grupo; el grupo puede salir con otro número en vuestro ordenador):

	grupo 0	grupo 1	grupo 2	grupo 3
`edad`	34,6	40,3	57,2	29,3
`gasto_medio_dia`	120,4	95,2	211,4	71,2
`noches_medias`	2,6	7,4	8,1	6,1
`reservas_anio`	4,0	1,3	1,6	2,1
`antelacion_media`	22,2	110,3	90,3	59,7
`pct_gasto_actividades`	41,2	26,0	20,0	74,6
`viaja_con_ninos`	0,1	0,9	0,0	0,0
`n_huespedes`	194	402	317	287
`pct`	16	34	26	24

4. **Comparar con la media: el mapa de calor.** Nueva celda:

```
perfil_z = pd.DataFrame(km.cluster_centers_, columns=variables)
fig, ax = plt.subplots(figsize=(9, 3.5))
sns.heatmap(perfil_z, annot=True, fmt=".1f", cmap="RdBu_r", center=0, ax=ax)
ax.set_title("Perfil de cada grupo (en desviaciones típicas respecto a la media)")
ax.set_ylabel("Grupo")
plt.show()
```

Qué hace cada línea:

- `km.cluster_centers_`: el centro de cada grupo, con los datos ya escalados; `pd.DataFrame(…, columns=variables)` los pone en una tabla con nombres.
- `sns.heatmap(…)`: la pinta. `annot=True` escribe el número en cada casilla y `center=0` pone el blanco en la media.

Un **0** es «en la media»; un **1,2**, «bastante por encima»; un **−1,7**, «muy por debajo». Los números grandes (en rojo o azul fuerte) son los que definen al grupo. Por ejemplo, el grupo de 402 tiene un 1,2 en `viaja_con_ninos` y el de 287, un 1,5 en `pct_gasto_actividades`.

5. **Poner nombre a cada segmento.** Cada nombre se apoya en **dos cifras** de su perfil. Con los datos de esta ejecución:

Grupo	Nombre	Tamaño	Las dos cifras que lo justifican	Acción comercial (ejemplo)
1	Familias de larga estancia	402 (34 %)	9 de cada 10 viajan con niños y reservan con 110 días de antelación	Pack «Niños gratis en el parque acuático» en la reserva
2	Mayores de alto gasto	317 (26 %)	57 años de media y 211 € de gasto al día	Mejora de habitación y experiencias tranquilas, con reserva anticipada
3	Jóvenes aventureros	287 (24 %)	29 años de media y el 75 % de su gasto en actividades	Newsletter solo con excursiones y deportes, con precio de grupo
0	Escapadas frecuentes	194 (16 %)	4 reservas al año y estancias de 2,6 noches	Oferta de última hora: reservan con unos 22 días de antelación

   Los números de grupo los pone el algoritmo y pueden salir permutados en otro ordenador. Mirad vuestro `perfil` y poned cada nombre donde corresponda: **no copiéis los nombres de aquí sin comprobarlos con vuestras cifras**. La IA puede sugerir nombres, pero un nombre que no cuadra con vuestras cifras no vale. Cada **acción** dice qué enseñar, cuándo y por qué canal, y tiene que servir a Lucía de verdad.

6. **Guardar los nombres y comprobar los tamaños.** Nueva celda, con **vuestros** nombres (la clave es el número de grupo, sin comillas):

```
nombres_segmentos = {0: "Escapadas frecuentes", 1: "Familias de larga estancia",
                     2: "Mayores de alto gasto", 3: "Jóvenes aventureros"}
perfil["nombre"] = pd.Series(nombres_segmentos)
print(perfil[["nombre", "n_huespedes", "pct"]])
print("Suma de tamaños:", perfil["n_huespedes"].sum())
```

   Qué hace cada línea:

   - `nombres_segmentos = {0: "…", …}`: un diccionario `número de grupo: nombre`. Las comillas rodean cada nombre; los dos puntos separan la clave y el nombre.
   - `perfil["nombre"] = pd.Series(nombres_segmentos)`: añade a la tabla una columna con el nombre de cada grupo.
   - La última línea suma los tamaños: tiene que salir **1200**.

   Después, en una celda de texto, apuntad una fila por segmento: nombre, tamaño, perfil con dos cifras y acción. Es lo que pasaréis a la ficha en el paso 5.

Qué deberíais ver. Un mapa con un color por segmento y el % de varianza en los ejes (aquí, 43 % y 26 %, y 68,7 % entre los dos), una tabla `perfil` con la media de cada variable, el tamaño y el porcentaje de cada grupo, el mapa de calor y, por segmento, un nombre, dos cifras y una acción. La suma de tamaños es 1200.

Hecho cuando: el mapa está dibujado y cada segmento tiene nombre, tamaño, perfil con cifras y una acción.

Si algo falla

Veis…	Qué significa	Qué hacer
`NameError: name 'PCA' is not defined` (o `'sns'`, `'X'`)	No se han ejecutado las celdas del paso 1.	Ejecutad desde arriba, en orden.
`KeyError: 'grupo'`	Aún no se ha ejecutado la celda del apartado 1 de este paso.	Ejecutadla antes del perfil y del mapa.
Aparece una columna `grupo` en `perfil` o en el mapa de calor	Se ha vuelto a ejecutar la celda `variables = …` **después** de crear `grupo`, y `grupo` se ha colado como variable.	*Entorno de ejecución → Reiniciar y ejecutar todo*: así `variables` se crea antes que `grupo`.

Veis…	Qué significa	Qué hacer
Los grupos del mapa se mezclan	Es posible: con 2 componentes solo se ve el 69 % de la información.	Decidlo en la ficha. No cambiéis los grupos para que «se vean mejor».
La columna `nombre` sale con `NaN` en algún grupo	Falta ese número en el diccionario o las claves llevan comillas (`"0"`).	Las claves son números sin comillas: `{0: "…", 1: "…"}`.
Vuestros grupos y tamaños no son los de aquí	El algoritmo numera los grupos a su manera y, con otra versión de scikit-learn, los números varían un poco.	Es normal. Seguid **vuestro** `perfil` y poned los nombres según vuestras cifras.

Paso 4 · Recomendador de experiencias (día 2, 30 minutos)

1. Separar valoraciones de entrenamiento y de prueba.
2. Montar al menos dos métodos y evaluarlos con RMSE y precision@3.
3. Programar `recomendar(id_usuario, n)`.
4. Probarla con 3 usuarios.

En sencillo. TurisData vende 25 experiencias (excursiones, talleres, deportes) y 300 clientes han puntuado algunas de 1 a 5 estrellas. Recomendar es como un librero que sugiere lecturas. Para saber si es bueno, le **escondéis** una parte de las opiniones (el 20 %) y miráis si las adivina, igual que un examen con preguntas que no ha visto. Probaréis dos libreros: uno que recomienda **lo más valorado por todos** (la estantería de los más vendidos) y otro que mira **qué experiencias gustan a la misma gente** («a quien le gustó el Teide, también le gustó…»). Y los mediréis de dos formas: cuánto se equivocan en la nota y si acierta la lista corta de tres. No hace falta entender las fórmulas: están escritas, y lo importante es saber qué mide cada cosa.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Matriz usuario × experiencia	Una tabla con un usuario por fila y una experiencia por columna; en cada casilla, la nota que dio.
`NaN`	Una casilla vacía: ese usuario no ha valorado esa experiencia. Aquí, el 76 % de las casillas están vacías.
Entrenamiento y prueba	El 80 % de las valoraciones, que usa el recomendador, y el 20 % escondido, con el que se le examina.
Popularidad	Recomendar a todos lo mejor valorado de media. Es la referencia que hay que batir, y la única opción con un huésped nuevo.
Item-item	Recomendar mirando qué experiencias son «parecidas» (las puntúa por encima o por debajo de lo normal la misma gente).
RMSE	«De media, ¿en cuántas estrellas se equivoca?». Cuanto **más bajo**, mejor.
precision@3	De las 3 experiencias recomendadas, qué fracción le gustó de verdad (4 estrellas o más en la prueba). Cuanto **más alta**, mejor.
Arranque en frío	Cuando no hay datos de un usuario nuevo y no se puede personalizar.
`-inf`	«Menos infinito»: un valor tan bajo que esa experiencia nunca saldrá entre las mejores. Sirve para ocultar lo ya valorado.

Cómo se hace

1. **Cargar los datos, esconder el 20 % y montar la matriz.** Nueva celda:

```
 from sklearn.metrics.pairwise import cosine_similarity

valoraciones = pd.read_csv(dato("valoraciones_experiencias.csv"))
experiencias = pd.read_csv(dato("experiencias.csv"))

prueba = valoraciones.sample(frac=0.2, random_state=SEMILLA)
entrenamiento = valoraciones.drop(prueba.index)

R = entrenamiento.pivot(index="id_usuario", columns="id_experiencia", values="valoracion")
R = R.reindex(index=range(1, 301), columns=range(1, 26))
M = R.notna().values
media_global = entrenamiento["valoracion"].mean()
print(valoraciones.shape, experiencias.shape, len(entrenamiento), len(prueba))
print("Matriz:", R.shape, "· casillas con dato:", f"{M.mean():.0%}")
```

   Qué hace cada línea:

   - `from sklearn.metrics.pairwise import cosine_similarity`: trae la herramienta que mide «lo parecidas que son» dos columnas (la usaréis en el método 2).
   - `valoraciones = …` y `experiencias = …`: leen las valoraciones (usuario, experiencia, nota) y el catálogo (nombre, categoría, isla).
   - `prueba = valoraciones.sample(frac=0.2, random_state=SEMILLA)`: **esconde** el 20 % de las valoraciones, elegidas al azar (con la semilla, siempre las mismas). `entrenamiento = valoraciones.drop(prueba.index)` se queda con el resto.
   - `R = entrenamiento.pivot(…)`: construye la matriz usuario × experiencia con **solo** el entrenamiento. `reindex(…)` asegura 300 usuarios (1 a 300) y 25 experiencias (1 a 25).
   - `M = R.notna().values`: una matriz de «sí/no» que marca dónde hay valoración. `media_global` es la nota media de todo el entrenamiento.

   Sale `(2232, 3) (25, 4) 1786 446` y `Matriz: (300, 25) · casillas con dato: 24%`.

2. **Las dos reglas para medir.** Nueva celda:

```
fila_p = prueba["id_usuario"].values - 1
col_p = prueba["id_experiencia"].values - 1
real_p = prueba["valoracion"].values

def evaluar_rmse(P):
    pred = np.clip(P[fila_p, col_p], 1, 5)
    return float(np.sqrt(np.mean((pred - real_p) ** 2)))

relevantes = prueba[prueba["valoracion"] >= 4]

def precision_k(P, k=3):
    aciertos = []
    for u, g in relevantes.groupby("id_usuario"):
        puntuacion = P[u - 1].astype(float).copy()
        puntuacion[M[u - 1]] = -np.inf
        recomendadas = np.argsort(-puntuacion)[:k] + 1
        aciertos.append(len(set(recomendadas) & set(g["id_experiencia"])) / k)
    return float(np.mean(aciertos))

print("Usuarios evaluables:", relevantes["id_usuario"].nunique())
```

   Qué hace cada línea:

   - `fila_p`, `col_p`, `real_p`: dónde están las valoraciones escondidas y qué nota tienen. Se resta 1 porque Python cuenta desde 0.
   - `def evaluar_rmse(P):`: recibe una matriz de notas previstas `P` y devuelve el **RMSE** en las valoraciones escondidas (`np.clip` mantiene la nota entre 1 y 5).
   - `relevantes = …`: las valoraciones escondidas de 4 estrellas o más: «lo que le gusta» a cada usuario.
   - `def precision_k(P, k=3):`: para cada usuario toma su fila de `P`, **oculta lo que ya valoró** con `-np.inf`, se queda con las `k` mejores (`np.argsort` ordena; `+ 1` pasa de posición a número de experiencia) y cuenta qué fracción estaba en lo que le gusta.

   Sale `Usuarios evaluables: 115`. **Copiadlas sin cambiar nada**: son las reglas del examen.

3. **Método 1: popularidad.** Nueva celda:

```
media_item = R.mean()
P_pop = np.tile(media_item.values, (300, 1))
print(f"Popularidad · RMSE = {evaluar_rmse(P_pop):.3f} · precision@3 = {precision_k(P_pop):.3f}")
top = media_item.sort_values(ascending=False).head(5)
print(experiencias.set_index("id_experiencia").loc[top.index, "nombre"].to_frame().assign(nota=top.round(2)))
```

   Qué hace cada línea:

   - `media_item = R.mean()`: la nota media de cada una de las 25 experiencias, con el entrenamiento.
   - `P_pop = np.tile(media_item.values, (300, 1))`: repite esa lista en 300 filas: **todos los usuarios reciben la misma predicción**.
   - Las dos últimas líneas escriben las 5 experiencias mejor valoradas con su nombre.

   Sale `Popularidad · RMSE = 1.029 · precision@3 = 0.125` y las cinco más valoradas: Parque Timanfaya (4,43), Visita a Garachico (4,21), Observación de cetáceos (3,93), Parapente (3,82) y Astronomía en el Roque (3,77).

4. **Método 2: item-item, y la tabla comparativa.** Nueva celda:

```
media_item_v = media_item.fillna(media_global).values
C = np.where(M, R.values - media_item_v[None, :], 0.0)
S = cosine_similarity(C.T)
np.fill_diagonal(S, 0)
P_item = media_item_v[None, :] + (C @ S.T) / (M.astype(float) @ np.abs(S).T + 1e-9)

metodos = {"Popularidad": P_pop, "Item-item": P_item}
tabla = pd.DataFrame({
    "RMSE": {n: evaluar_rmse(P) for n, P in metodos.items()},
    "precision@3": {n: precision_k(P) for n, P in metodos.items()},
}).round(3)
tabla
```

   Qué hace cada línea:

   - `C = …`: para cada casilla con dato, cuánto puntuó el usuario **por encima o por debajo** de la nota media de esa experiencia (0 si no la valoró).
   - `S = cosine_similarity(C.T)`: una tabla 25 × 25 con el parecido entre experiencias (1, van a la par; 0, nada que ver; −1, opuestas). `np.fill_diagonal(S, 0)` evita que una experiencia se ayude a sí misma.
   - `P_item = …`: la nota prevista de cada usuario para cada experiencia: su nota media más lo que ese usuario puntuó en las experiencias parecidas. No hace falta entender la fórmula: es la de `S09_03`.
   - `metodos` y `tabla`: juntan los dos métodos en una tabla con su RMSE y su precision@3.

   Sale esta tabla:

	RMSE	precision@3
Popularidad	1.029	0.125
Item-item	0.895	0.171

   Si en clase hicisteis el método por contenido (`S09_03`, apartado 4), podéis añadir `"Contenido": P_cont` a `metodos`: sale con RMSE 1.066 y precision@3 0.130, **sin mejorar** a la popularidad. Los nombres de las variables son los mismos que en `S09_03`, así que podéis copiar de ahí.

5. **Elegir el método con una frase.** Comparad con prudencia: con solo 115 usuarios evaluables, una diferencia de 0,01 es ruido y no se vende como mejora. Aquí, 0,171 frente a 0,125 en precision@3 y 0,895 frente a 1,029 en RMSE son diferencias claras, aunque la prueba sea pequeña. Escribid en una celda de texto una frase como esta, con **vuestras** cifras: *«Elegimos item-item porque su precision@3 es 0,171 frente a 0,125 de la popularidad y su RMSE es 0,895 frente a 1,029.»*

6. **Programar `recomendar` y probarla.** Nueva celda:

```
def recomendar(id_usuario, n=3, P=P_item):
    puntuacion = P[id_usuario - 1].astype(float).copy()
    puntuacion[M[id_usuario - 1]] = -np.inf
    ids = np.argsort(-puntuacion)[:n] + 1
    res = experiencias.set_index("id_experiencia").loc[ids, ["nombre", "categoria", "isla"]].copy()
    res["nota_prevista"] = np.clip(P[id_usuario - 1][ids - 1], 1, 5).round(2)
    return res

for u in (1, 2, 3):
    print("Usuario", u)
    print(recomendar(u))
    vistas = set(np.where(M[u - 1])[0] + 1)
    print("Recomendadas que ya había valorado:", set(recomendar(u).index) & vistas)
```

   Qué hace cada línea:

   - `def recomendar(id_usuario, n=3, P=P_item):`: una función con tres entradas: el usuario, cuántas experiencias (3 si no se dice) y la matriz de notas previstas (item-ítem si no se dice).
   - `puntuacion = P[id_usuario - 1]…` y `puntuacion[M[id_usuario - 1]] = -np.inf`: toma la fila del usuario y pone `-inf` en lo que **ya valoró**, para que no se recomiende.
   - `ids = np.argsort(-puntuacion)[:n] + 1`: ordena de mayor a menor nota prevista y se queda con las `n` primeras.
   - `res = …loc[ids, …]` y `res["nota_prevista"] = …`: busca nombre, categoría e isla en el catálogo y añade la nota prevista.
   - El bucle `for u in (1, 2, 3)` la prueba con tres usuarios y comprueba que ninguna recomendada estaba ya valorada: tiene que salir `set()`, que significa «ninguna».

   Para el usuario 1 sale Parque Timanfaya (4,73), Ruta por Anaga (4,14) y Zoo y jardines (3,87); para el 2, Observación de cetáceos (4,55), Astronomía en el Roque (4,51) y Parapente (3,90); y para el 3, Visita a Garachico (5,00), Ruta del queso majorero (4,17) y Ruta por Anaga (3,99). Son distintas entre sí. Solo existen los usuarios **1 a 300**.

7. **Pensar en el huésped nuevo.** Un huésped que no ha valorado nada no tiene fila en la matriz: es el **arranque en frío**. Aquí lo único posible es la popularidad: enseñarle las mejores de `top` (Parque Timanfaya, Visita a Garachico, Observación de cetáceos). Dejadlo escrito en una celda de texto: irá a la ficha.

Qué deberíais ver. La tabla comparativa de los dos métodos (item-ítem mejor que la popularidad en las dos medidas), la frase de elección y tres listas de tres experiencias, distintas entre sí y sin nada ya valorado. Las cifras pueden variar un poco con otras versiones de las bibliotecas.

Hecho cuando: el recomendador está evaluado con valoraciones escondidas, `recomendar` funciona con 3 usuarios y no propone nada ya valorado.

Si algo falla

Veis…	Qué significa	Qué hacer
`NameError: name 'M' is not defined` (o `'R'`, `'P_pop'`, `'cosine_similarity'`)	Las celdas se han ejecutado en desorden o Colab se reinició.	Ejecutad las celdas de este paso de arriba abajo, empezando por la primera (la del `import`).
`KeyError: 'valoraciones'`	En `pivot` habéis escrito `values="valoraciones"`; la columna es `valoracion`.	Escribid `values="valoracion"`, en singular.
`IndexError: too many indices for array: array is 1-dimensional, but 2 were indexed`	`P_pop` no se ha repetido con `np.tile`: es una sola fila.	Usad `np.tile(media_item.values, (300, 1))`.

Veis…	Qué significa	Qué hacer
`IndexError: index 300 is out of bounds for axis 0 with size 300`	Habéis pedido `recomendar(301)`: solo hay usuarios 1 a 300.	Probad con un número entre 1 y 300. Un huésped nuevo se resuelve con la popularidad.
`recomendar(0)` no da error, pero devuelve recomendaciones	El usuario 0 no existe; Python cuenta `0 - 1 = -1` desde el final y devuelve las del usuario 300.	Los usuarios empiezan en 1.
`recomendar` propone algo que el usuario ya valoró	Falta `puntuacion[M[id_usuario - 1]] = -np.inf`.	Añadid esa línea antes de ordenar.
Todos los usuarios reciben casi las mismas experiencias	La función usa `P_pop` (la popularidad), no `P_item`.	Llamadla sin `P=` (usa item-item) o con `P=P_item`.

Paso 5 · Ficha, IA responsable, revisión y entrega (día 2, 30 minutos)

1. Rellenar la ficha de segmentos y la nota de IA responsable.
2. Pedir a alguien que no escribió esa parte que revise con la rúbrica y anote el resultado en la ficha.
3. Dejar el cuaderno ejecutable y subir los ficheros a `sprint-09/`.
4. Ensayar 3 minutos de review.

En sencillo. Lucía no va a abrir vuestro cuaderno: lo que lee es la **ficha**, una página que le cuenta quiénes son sus clientes y qué hacer con ellos, como el prospecto de un medicamento. Casi no hay código: se escribe **texto** con las cifras que ya tenéis. Además hay que contar con honestidad qué riesgos tiene usar estos grupos y estas recomendaciones, y dejar el cuaderno de modo que otra persona lo repita. Después, alguien del equipo que no escribió esa parte lo revisa con la rúbrica, con ojos frescos, y entre todos subís el trabajo a GitHub. No hace falta saber Markdown: la plantilla trae las tablas ya montadas.

Palabras nuevas de este paso

Palabra	Qué significa, en sencillo
Ficha de segmentos	La página para el cliente: método y `k`, mapa, una fila por segmento, recomendador, riesgos y límites. Es el fichero `ficha_segmentos.md`.
Markdown (`.md`)	Texto normal con marcas sencillas: `#` para los títulos y barras verticales para separar las columnas de una tabla.
Experiencia estrella	La experiencia que mejor encaja con un segmento y que recomendaríais a ese grupo.
Supuesto	Algo que damos por cierto sin poder comprobarlo con los datos. Se escribe como tal.
Sesgo de popularidad	Que se recomienden siempre las mismas experiencias (las más conocidas) y las demás casi nunca salgan.
Discriminación	Tratar peor a alguien por su edad, por viajar con niños, etc. (por ejemplo, subiéndole el precio).
Rúbrica	La tabla con que se evalúa el reto: cuatro criterios, cada uno de 1 a 4.
Commit	Guardar en GitHub un cambio con un mensaje corto y vuestro nombre. Sin pulsar **Commit changes** no se guarda nada.

Cómo se hace

1. **Crear `ficha_segmentos.md` con la plantilla.** Abrid la plantilla y pulsad **Raw**, `Ctrl + A`, `Ctrl + C`. Después, en el repositorio del equipo, entrad en la carpeta `sprint-09`, **Add file → Create new file**, nombre `ficha_segmentos.md`, y pegad. Rellenad cada sección **con vuestras cifras**, sin dejar los `_____`:

Sección	Qué escribís
Cabecera	Equipo, método (k-means; tachad lo demás), `k`, silueta (aquí, 0,431 con `k` = 4) y la frase de las dos razones del paso 2
Mapa de segmentos	La imagen `mapa_pca.png` (se baja del panel de archivos de Colab y se sube a `sprint-09`) con una línea como «Mapa: ver el fichero `mapa_pca.png`», o decid dónde está en el cuaderno
Segmentos	Una fila por segmento: nombre, tamaño (n y %), perfil con cifras, acción y experiencia estrella. La tabla `perfil` y la de nombres del paso 3 ya traen casi todo
Recomendador	Los dos métodos con RMSE y precision@3, el elegido y por qué, un ejemplo con un usuario (aquí, el usuario 1 → Parque Timanfaya, 4,73) y qué haría el sistema con un huésped nuevo (popularidad)
IA responsable	Cinco líneas, con **al menos 2 riesgos específicos** de este caso y su medida (apartado 3)
Limitaciones	Datos sintéticos, comportamiento de un año, 4 a 11 valoraciones por persona, el mapa solo conserva el 69 % y la prueba del recomendador tiene 115 usuarios. Añadid las vuestras

2. **Elegir la experiencia estrella de cada segmento.** Ojo: los datos son sintéticos y las 300 personas de las valoraciones **no son** las de `huespedes.csv`, así que **no hay un cruce real** entre los dos ficheros. Elegid la experiencia razonando con el perfil del segmento, apoyadla en lo que recomienda vuestro prototipo y **anotadla como supuesto**. Ejemplos con los datos de esta ejecución (la cuarta es para vosotros):

Segmento	Experiencia estrella	Por qué (y nota media en el prototipo)
Familias de larga estancia	Parque acuático (familia)	9 de cada 10 viajan con niños (3,68)
Jóvenes aventureros	Parapente (aventura)	El 75 % de su gasto es en actividades (3,82)
Mayores de alto gasto	Visita a Garachico (cultura)	Gastan 211 € al día y casi nada en actividades: plan tranquilo (4,21)
Escapadas frecuentes	…	…

Mirad la nota media de la experiencia: no pongáis de estrella una que el prototipo valora mal (el Spa y termas tiene un 1,70). Escribid: «Supuesto: los usuarios que valoraron no son los huéspedes de este segmento».

3. **La nota de IA responsable (5 líneas).** Elegid los riesgos que valen para **vuestro** caso, cada uno con una medida. Ejemplos:

```
- Riesgo 1 (segmentar): discriminar, si se usa la edad o «viaja con niños» para cobrar precios distintos. Medida: usar los segmentos para decidir qué comunicar, no para variar precios.
- Riesgo 2 (recomendar): sesgo de popularidad. Medida: reservar un hueco de cada lista a una experiencia menos conocida.
- Datos personales implicados y base legal: …
```

   Si queréis **comprobar** el riesgo 2 con datos (es un extra), añadid esta celda al cuaderno, después de `recomendar`:

```
todas = [i for u in range(1, 301) for i in recomendar(u, 3).index]
print(len(set(todas)), "experiencias distintas recomendadas, de 25")
print(pd.Series(todas).value_counts().head(3))
```

   Qué hace cada línea:

   - La primera recorre los 300 usuarios, pide a `recomendar` tres experiencias para cada uno y las junta en una lista (`todas`, con 900 números).
   - `len(set(todas))`: `set` elimina los repetidos y `len` cuenta los que quedan: cuántas experiencias **distintas** se han recomendado.
   - `pd.Series(todas).value_counts().head(3)`: las tres que más se repiten y cuántas veces.

   Con item-item sale `18 experiencias distintas recomendadas, de 25`, y la 11 (Parque Timanfaya) se repite 160 veces, la 22 (Garachico), 144 y la 24 (Parapente), 106. Con la popularidad (`P=P_pop`) solo saldrían 8. Es una cifra honesta para la ficha: el sesgo existe, aunque item-item lo reduce.

4. **Declarar la IA en el cuaderno.** En una celda de texto, con este molde: «Usamos la IA para …; comprobamos …; cambiamos …». Si no la usasteis, decidlo. **Nada de datos personales ni claves** en una IA.
5. **La revisión con la rúbrica (10 minutos).** La hace **una persona que no escribió esa parte**. Sigue los cinco pasos de «Cómo se revisa» de entrega y rúbrica: ejecutar el cuaderno con *Reiniciar y ejecutar todo*, leer la ficha como si fuera Lucía (¿se entiende qué hacer con cada segmento? ¿los tamaños suman 1200?), mirar el cuaderno (¿se escala antes de agrupar?, ¿`recomendar` no devuelve nada ya valorado?), **comprobar que cada nombre sigue encajando con su fila de `perfil`** y poner un nivel de 1 a 4 por criterio con **una frase de prueba**, por ejemplo: «3 · el cuaderno se ejecuta entero y salen k = 4 y los mismos tamaños, pero falta probar `recomendar` con un tercer usuario». Sin prueba escrita, el nivel no vale. Se apunta en la sección «Revisión con la rúbrica» del final de `ficha_segmentos.md` (borrad el texto entre `<…>`). Si hay tiempo, arreglad lo más fácil; lo demás queda como «lo primero que mejoraríamos».
6. **Dejar el cuaderno y subir los ficheros (la vía del curso, desde el navegador).** En Colab: *Entorno de ejecución → Reiniciar y ejecutar todo* (ninguna celda debe dar error) y *Archivo → Descargar → Descargar .ipynb*; bajad también `mapa_pca.png` desde el panel de archivos (clic derecho → *Descargar*). Después, seguid Cómo entregar en GitHub: (1) abrid el repositorio del equipo y **entrad primero en la carpeta `sprint-09`**; (2) *Add file → Upload files* y arrastrad los ficheros; (3) escribid un mensaje corto («Añado segmentación y recomendador») y pulsad **Commit changes**. Un reparto posible: la persona A sube el cuaderno, la B la ficha y el mapa («Añado ficha de segmentos»), y cada persona sube su autoevaluación. Nombres de archivo **sin espacios ni tildes**.
7. **Ensayar la review (3 minutos).** Preparad qué enseñaréis: **vuestros segmentos (nombre y acción)** y **una recomendación concreta para un usuario**. Decidid quién cuenta qué.

Qué deberíais ver. En la carpeta `sprint-09` del repositorio, el cuaderno y la ficha del equipo (y el mapa, si lo habéis puesto) y una hoja de autoevaluación por persona (**2 + N**); y en el historial (el icono del reloj), *commits* de todas las personas del equipo.

Hecho cuando: otra persona podría repetir el cuaderno de arriba abajo, la sección «Revisión con la rúbrica» está rellena y cada persona ha subido algo con su cuenta.

Después, 40 minutos de entrega, review y retrospectiva: cada persona rellena y sube su autoevaluación y coevaluación a `sprint-09/`, y el equipo presenta en 3 minutos **sus segmentos (nombre y acción)** y **una recomendación concreta para un usuario**. Se comparan los `k` elegidos y se discute: *¿es el mismo cliente aunque le pongamos otro nombre?*

Si algo falla

Veis...	Qué significa	Qué hacer
Los tamaños de la ficha no suman 1200	Alguna cifra está mal copiada.	Copiadlas de la tabla `perfil` y comprobad con `perfil["n_huespedes"].sum()`.
No encontráis el mapa al leer la ficha	El fichero `mapa_pca.png` no está en `sprint-09` o la ficha no dice dónde está.	Subidlo a la misma carpeta (nombre exacto, en minúsculas) y escribid en la ficha «Mapa: ver el fichero `mapa_pca.png`».
Las tablas de la ficha se ven como líneas de texto con barras verticales	Se ha borrado alguna de las barras de la tabla o la línea de guiones que hay bajo la cabecera.	Pulsad **Preview** en GitHub, comparad con la plantilla y reponed lo que falte.
Tras *Reiniciar y ejecutar todo*, los nombres no encajan con los perfiles	Los números de grupo pueden salir en otro orden con otra versión de las bibliotecas, y el diccionario `nombres_segmentos` va por número.	Mirad el `perfil` y corregid el diccionario según vuestras cifras. Dejad escrita la versión en el cuaderno.
Al *Reiniciar y ejecutar todo*, una celda da error en rojo	Una celda depende de otra que está debajo o que falla.	Leed el error de la **primera** celda en rojo, corregidla y repetid.
Subisteis un fichero a la raíz del repositorio y no a `sprint-09`	No entrasteis primero en la carpeta.	Abrid el fichero → icono del lápiz → delante del nombre escribid `sprint-09/` → **Commit changes**.
No sabéis qué nivel poner en la rúbrica	Los niveles están descritos en entrega y rúbrica.	Poned el nivel cuyo texto **se cumple entero** (si algo se cumple a medias, es el de abajo) y escribid la prueba.

Ayuda 1 · Cómo usar la IA en este reto

La IA puede ahorrar tiempo, pero **inventa con mucha seguridad**: un nombre de segmento que suena bien pero no refleja las cifras es un error. Reglas:

- **Todo lo que dé, lo comprobáis y lo entendéis.** Si el docente pregunta por una celda, tenéis que saber explicarla; en la review puede preguntar a cualquiera.
- **Los datos mandan:** si un nombre o una conclusión de la IA no cuadra con vuestras cifras, no vale.
- **Declaradlo** en el cuaderno: dónde la habéis usado y qué habéis comprobado.
- **Nada de datos personales ni claves** en una IA.

Para qué	Ejemplo de petición	Qué comprobáis
Entender un concepto	«Explícame k-means y la silueta con una analogía de personas en una fiesta»	Que podéis explicarlo con vuestras palabras
Entender PCA	«¿Qué significa que PCA conserve el 60 % de la varianza? Dame un ejemplo sin fórmulas»	Que vuestro gráfico dice lo mismo
Poner nombres a los segmentos	«Dado este perfil medio: *[tabla]*, dame 5 nombres comerciales cortos y evocadores»	Que el nombre refleja las **cifras** del segmento, no una invención
Pensar en riesgos	«¿Qué riesgos tiene recomendar experiencias según la edad?»	Elegid los que aplican a **vuestro** caso y dad una medida real

Ayuda 2 · Errores frecuentes (y cómo evitarlos)

Error	Por qué ocurre	Cómo evitarlo
Agrupar sin escalar	Se olvida	Escalad **antes** del clustering
Elegir `k` solo con la silueta	«Es lo que dice el número»	Añadid el criterio de negocio
Poner nombres sin mirar los datos	Suena bien	Cada nombre debe justificarse con **dos cifras** del perfil
Recomendar lo que el usuario ya ha valorado	No se filtra	La función debe excluirlas
Evaluar el recomendador con los mismos datos que usó	«Es lo más fácil»	Probad con valoraciones **escondidas**
Nota de IA responsable genérica	Prisa	Dos riesgos **concretos** de este caso, con medida
El cuaderno falla al ejecutarlo de nuevo	Celdas ejecutadas en desorden o semillas sin fijar	*Reiniciar y ejecutar todo* antes de subirlo y usar siempre la misma `SEMILLA`

Ayuda 3 · Si os atascáis

1. **Todos los segmentos se parecen:** revisad que habéis escalado y probad otras columnas; quizá algunas no aportan.
2. **El mapa PCA sale con todo mezclado:** es posible; decidlo en la ficha. Los grupos pueden estar separados en dimensiones que el 2D no recoge.
3. **El recomendador recomienda siempre lo mismo:** probablemente es un sesgo de popularidad; comentadlo como riesgo.
4. Repasad el cuaderno de ese paso y su código explicado; preguntad al equipo y, después, al docente. Más ayuda: Cómo se trabaja un reto.